

Conception Web

L1 S2 informatique

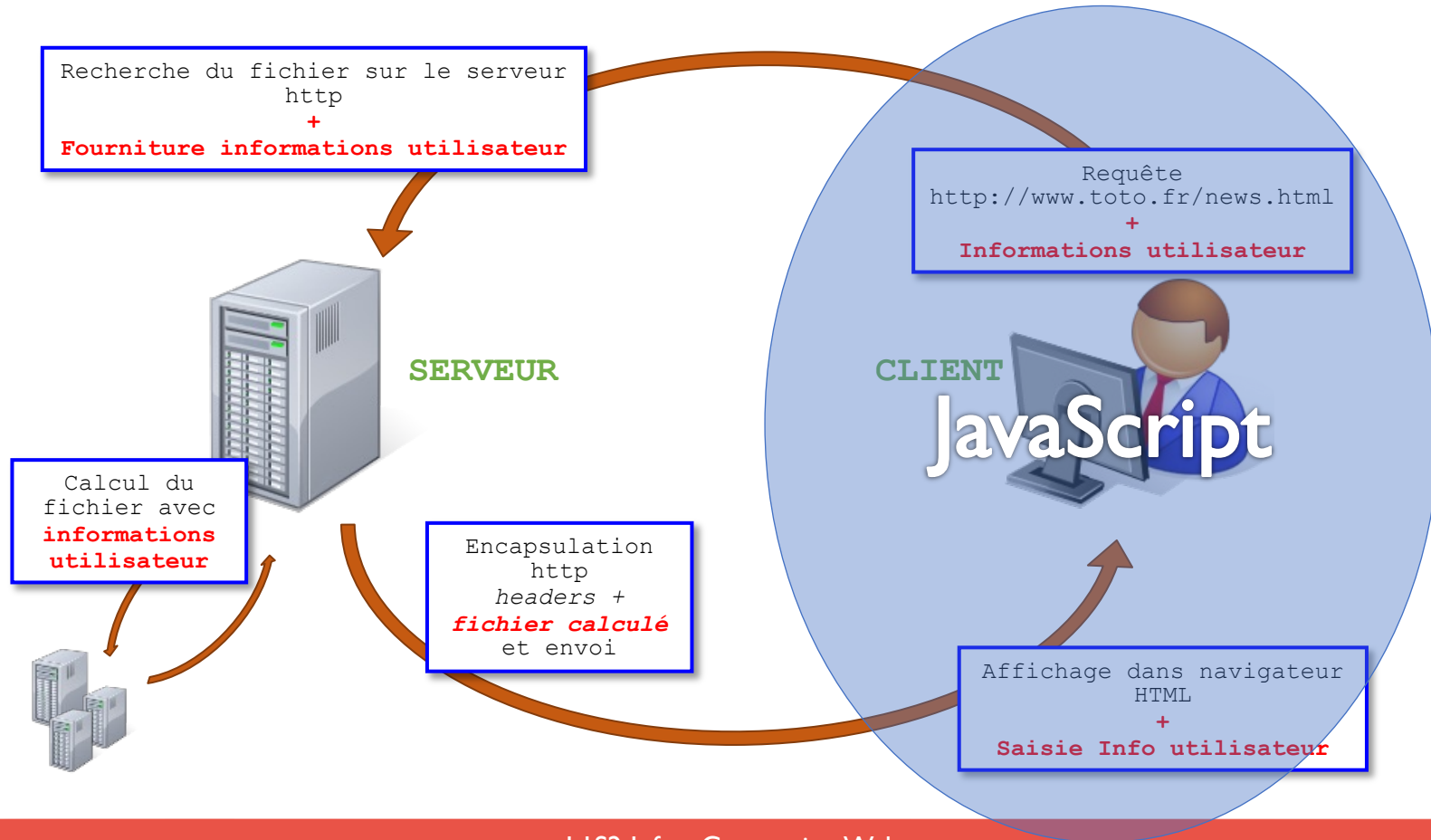
Fabrice Lefèvre

2026

CM 9

JavaScript

Interaction dynamique et JS



Quelques exemples

- Quelques exemples illustrant les capacités de Javascript :
 - Changement de propriétés @javascript_journuit.html
 - Menu avec ancrs @javascript_ancres.html
 - Technique du rollover @javascript_roll-over.html
@javascript_clignotant.html
 - Combinaison formulaire HTML @js_radiocheckbox.html
- Attention : comportements peut être encore différents selon les réglages des navigateurs (eg pour ouverture de fenêtres)

Exemple @javascript_journuit.html

```
1 <html>
2 <head>
3 <title>onclick</title>
4
5 <script language="javascript">
6   function change() {
7     document.body.style.color = 'yellow'
8     document.body.style.backgroundColor = 'grey'
9     document.form1.coul.value = 'gris'
10  }
11 </script>
12 </head>
13
14 <body>
15
16 <h1>onclick event</h1>
17 <br /><br />
18 blabla bla blabla blabla bla blabla bla blabla blabla bla blabla bla blabla blabla bla blabla
19 bla blabla blabla bla blabla bla blabla blabla bla blabla bla blabla blabla bla blabla
20 blabla bla blabla
21 bla blabla blabla bla blabla bla blabla blabla bla blabla bla blabla blabla bla blabla
22 blabla bla blabla
23 <br /><br />
24 <form name="form1">
25   <input type="text" name="coul">
26   <br /><br />
27   <input type="button" name="int" value="cliquez ici..." onclick="change()">
28 </form>
29 </body>
30 </html>
31
32
```

Exemple @javascript_clignotant.html

```
1 <html>
2 <head>
3 <title>blink</title>
4 <script>
5 function changeimage() {
6     setInterval(function(){
7         if (document.theatre.src.search("images/theatre-vert.gif") != -1) {
8             document.theatre.src='images/theatre-rouge.gif';
9         } else {
10             document.theatre.src='images/theatre-vert.gif';
11         }
12     }, 1000);
13 }
14 </script>
15 </head>
16
17 <body>
18 <h1>Défilement d'images automatique</h1>
19 
20 </body>
21 </html>
22
```

Caractéristiques de JavaScript

- Langage de script : donc interprété
- Profondément lié au langage HTML :
 - appel de JavaScript directement dans les balises HTML
 - totalité des balises HTML accessibles en JavaScript pour des manipulations diverses
 - grâce au DOM ! (on explique après)
 - gestion événementiel : le navigateur intercepte des événements associés à une balise et exécute le code JS associé (fonction de rappel, ou callback, ou event handler)
- Nom trompeur : syntaxe comparable à Java mais très simplifiée
- Modélisation objet light :
 - notion de classes très allégée
 - pas d'héritage entre classes

JavaScript : limites

- Code JavaScript interprété par le navigateur
- Possibilité de désactiver cette interprétation au niveau des navigateurs → code JavaScript totalement ignoré par le navigateur
- JavaScript ne doit pas supporter la responsabilité de sécuriser les interactions client/serveur
 - ex de la vérification de la syntaxe des adresses e-mail : si réalisée uniquement par du code JavaScript, possibilité de désactiver ce dernier et de renvoyer une fausse adresse pour « hacker » le serveur
 - mail toto@toto.fr ; cat /etc/passwd
- Tests de validation et sécurité des données peuvent être redondants sur le client et le serveur !

JavaScript : pros and cons

- **Avantages**

- bonnes possibilités d'interactivité
- langage simple
- facile à déployer (un simple éditeur texte et un navigateur qui le supporte)

- **Inconvénients**

- se désactive par l'utilisateur
- fonctionnement (encore) inégal entre les navigateurs
- script ouvert. Le code que vous écrivez étant visible par tous, vous ne devez pas écrire de code sensible, par exemple la méthode de validation d'un examen

Syntaxe (1)

- **Assez similaire au java mais pas strictement :**
 - sensible à la casse (majuscule/minuscule)
 - ";" non obligatoire en fin d'instruction (mais recommandé !)
 - commentaires : // ou /* commentaires */
- **Variables**
 - déclaration non obligatoire (mais recommandée !)
 - var ma_variable ;
 - Pas de type défini lors des déclarations, mais lors des affectations :
 - Nombre
 - Booléen
 - String
 - Objet (ex: array)
 - Fonction

Syntaxe (2)

- Instructions conditionnelles : if ... else, switch ... case
- Itérations (boucles) : for, for ... in, while, do ... while

- Gestion des fonctions :

```
function nom_fonction(par 1, par2 ...) {  
    par1 = par2;  
    return par1;  
}
```

- Gestion des objets :
 - appel au constructeur : variable = new Class()
 - accès aux propriétés par : variable.la_propriete
 - appel aux méthodes par : variable.la_methode(...)

CM 9 suite

JavaScript intégration

Insertion de codes JavaScript

- Différentes manières d'inclure du JavaScript dans une page HTML :
 - Au travers des balises d'insertion
`<SCRIPT> code JS </SCRIPT>`
 - Appel directement de fonctions JavaScript dans les balises HTML à l'aide des attributs « capteurs d'événements » (event handler)
 - [rare] Par le pseudo protocole "javascript" dans l'attribut href

Insertion par `<SCRIPT></SCRIPT>` (1)

- Balise `<SCRIPT></SCRIPT>` → insertion de code destiné à être exécuté par le navigateur
 - JavaScript ou autre
 - `<SCRIPT language="nom_langage" src="fichier.js" > code </SCRIPT>`
 - langage : JavaScript, JavaScript1.1, JavaScript1.2 (optionnel)
 - src : fichier pouvant contenir le code JavaScript à exécuter (ex : ensemble de fonctions).
 - extension : .js (configuration du serveur HTTP)
 - code : déclaration de fonctions ou de commandes JavaScript à exécuter
- **Placement dans le fichier HTML :**
 - on fait comme on veut
 - mais bonnes pratiques :
 - `<HEAD>...</HEAD>` → déclaration de fonctions stockées jusqu'à leur appel
 - `<BODY>...</BODY>` → exécution des commandes dès le chargement de la page

Insertion par <SCRIPT></SCRIPT> (2)

```
<HTML>
<HEAD></HEAD>
<BODY>
...
<SCRIPT>
  /* Permet d'afficher ce message sur la page HTML */
  document.write('Voici un appel à du code JavaScript directement exécuté dès le
chargement de cette page!');
</SCRIPT>
...
</BODY>
</HTML>
```



Insertion par <SCRIPT></SCRIPT> (2)

```
<HTML>
<HEAD>
  <SCRIPT>
    function afficheDial(message) {
      alert(message);
    }
  </SCRIPT>
</HEAD>
<BODY>
  ...
  <SCRIPT>
    afficheDial('ex de fonctions !');
  </SCRIPT>
  ...
</BODY>
```

Insertion de code entre
les balises
<HEAD>...</HEAD>

Insertion par <SCRIPT></SCRIPT> (3)

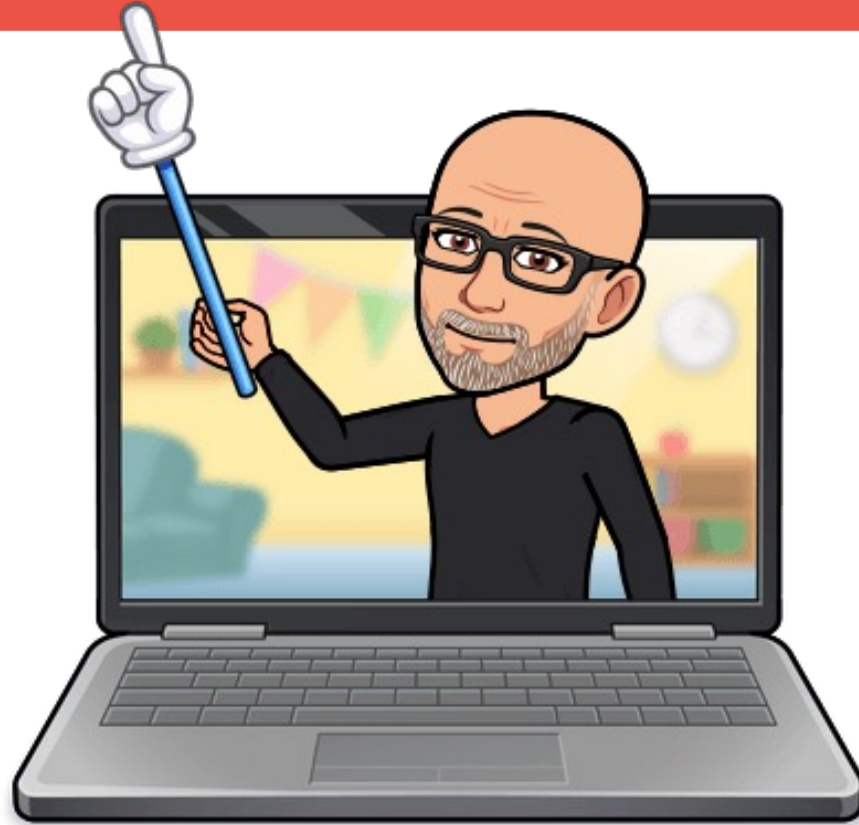
```
<HTML>
<HEAD>
  <SCRIPT SRC="my_fic.js"> </SCRIPT>
</HEAD>
<BODY>
...
  <SCRIPT>
    afficheDial('ex de fonctions !');
  </SCRIPT>
...
</BODY>
</HTML>
```

Fichier Javascript my_fic.js

```
function afficheDial(message){
  alert(message);
}
```

Insertion de code entre les balises <HEAD>...</HEAD> au travers d'un fichier externe

En pratique



Appels directs : attributs événements HTML

- Des événements associés directement aux balises HTML
@javascript_onchange.htm
 - un clic de souris sur un élément
 - sélection d'un champ d'un formulaire
 - le chargement d'une page
 - ...
- Permettent d'intercepter les actions utilisateurs (mais pas que)
- Permettent d'associer les scripts à exécuter lorsque ces événements sont détectés : *event handler*
- Accès direct par un attribut portant le nom de l'évènement

Connecter JavaScript et HTML

- Gestion des événements dans partie suivante
- Mais comment "accéder" aux parties du document HTML depuis un script JavaScript sans forcément passer par un événement d'une balise ?
 - le DOM !

DOM (1)

- Modèle Objet de Document, (ou DOM, Document Object Model) est un standard du W3C permettant l'accès aux documents HTML et XML.
- Permet principalement deux choses au développeur/concepteur :
 - fournit une représentation structurée du document
 - codifie la manière dont un script peut accéder à cette structure à travers les objets qu'elle contient
 - au niveau des propriétés
 - et des méthodes
- Donc, en pratique :
 - ➔ Permet de lier un document web en HTML à un langage de programmation ou de script

DOM (2)

- Il s'agit d'une interface qui permet de standardiser les moyens d'accès à une page Web à travers ses éléments.
 - ≠ application particulière,
 - ≠ produit spécifique
 - ≠ présentation propriétaire d'une page Web.
- **Toutes les propriétés et méthodes et tous les événements disponibles au développeur pour manipuler et créer des pages dynamiques sont organisés sous forme de hiérarchies d'objets**
 - le document en entier est un nœud document
 - chaque élément HTML est un nœud élément
 - chaque attribut HTML est un nœud attribut
 - le texte dans les éléments HTML sont des nœuds texte
 - les commentaires sont des nœuds commentaires
 - ...

Interlude...

- Peut-être utile de définir rapidement la notion d'objet en informatique ?
 - peut-être plus en 2025 (avec UE Objet dès le L1) ?
- Un objet du réel modélisé en informatique se résume rarement à un type simple de variable. Comment représenter un tel objet de façon efficace ?
- POO, Programmation Orientée Objet : ensemble de variables descriptives regroupées, et associées aux fonctions permettant leur manipulation
 - propriétés
 - méthodes (par exemple création et destruction : constructeur et destructeur)
- Classe → type informatique structuré
- Instance d'une classe → objet
- Mode de déclaration et de désignation simple (notation pointée) :
 - MaClasse monObjet
 - monObject.maPropriete
 - monObjet.maMethode(arguments)
- Encapsulation, héritage, polymorphisme...

Interlude...

- Ecrire le code décrivant une classe COMPTE comprenant les attributs suivants :
 - Numéro de Compte : Entier
 - Nom du Titulaire : Chaîne(30)
 - Solde : Réel

Interlude...

- Déclaration de la classe
- `compte=classe`
 - privé :
 - `//` cette partie sera inaccessible en dehors de la classe
 - `numéro : entier`
 - `nom : chaine(30)`
 - `solde : réel`
 - public :
 - `//` cette partie sera accessible et constitue l'interface de la classe
 - `procédure init(pnum:entier ; pnom:chaine(30))`
 - `procédure créditer(montant : réel)`
 - `procédure débiter(montant : réel)`
 - `fonction solde() : réel`

Propriétés

Méthodes

Interlude...

- **Description des méthodes**

- procédure compte.init(pnum:entier ; pnom:chaine(30))
début
 numéro:=pnum
 nom:=pnom
 Solde:=0
fin
- procédure compte.créditer(montant : réel)
début
 solde:=solde+montant
fin
- procédure compte.débiter(montant : réel)
début
 solde:=solde - montant
fin
- fonction compte.solde() : réel
début
 retourner(solde)
fin

Interlude

- Programme Exemple

- // déclaration d'un pointeur, pas d'instanciation
- var c1 : pointeur de compte
- Début
 - c1=nouveau(compte) // instanciation d'un compte
 - c1.init(1,"Dupont")
 - c1.créditer(1000)
 - c1.débiter(500)
- - afficher(c1.solde) //interdit : solde est en partie privée !
 - afficher(c1.solde()) // affiche 500
 - détruire(pc) // libération de la mémoire allouée
- Fin

DOM (2)

- Il s'agit d'une interface qui permet de standardiser les moyens d'accès à une page Web à travers ses éléments.
 - ≠ application particulière,
 - ≠ produit spécifique
 - ≠ présentation propriétaire d'une page Web.
- **Toutes les propriétés et méthodes et tous les événements disponibles au développeur pour manipuler et créer des pages dynamiques sont organisés sous forme de hiérarchies d'objets**
 - le document en entier est un nœud document
 - chaque élément HTML est un nœud élément
 - chaque attribut HTML est un nœud attribut
 - le texte dans les éléments HTML sont des nœuds texte
 - les commentaires sont des nœuds commentaires
 - ...

Interprétation DOM (1)

```
<TABLE>
<TBODY>
<TR>
<TD>ShadyGrove</TD>
<TD>Aeolian</TD>
</TR>
<TR>
<TD>Over the river, Charlie</TD>
<TD>Dorian</TD>
</TR>
</TBODY>
</TABLE>
```

Interprétation DOM (1bis)

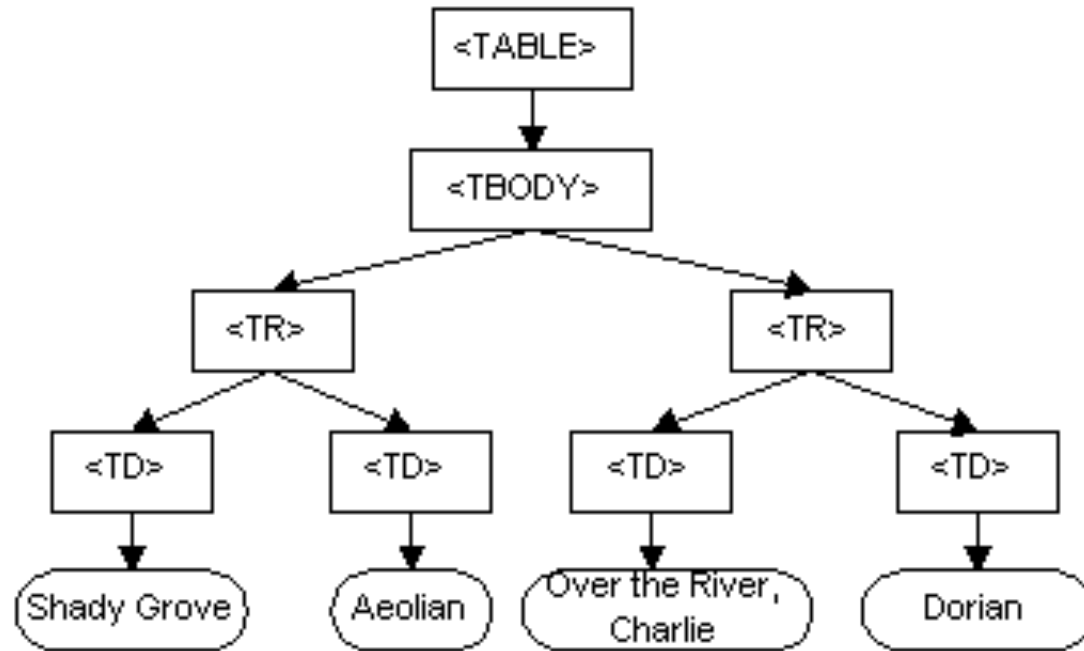
```
<TABLE>
  <TBODY>
    <TR>
      <TD>ShadyGrove</TD>
      <TD>Aeolian</TD>
    </TR>
    <TR>
      <TD>Over the river, Charlie</TD>
      <TD>Dorian</TD>
    </TR>
  </TBODY>
</TABLE>
```

Indentation rend lisible le code :

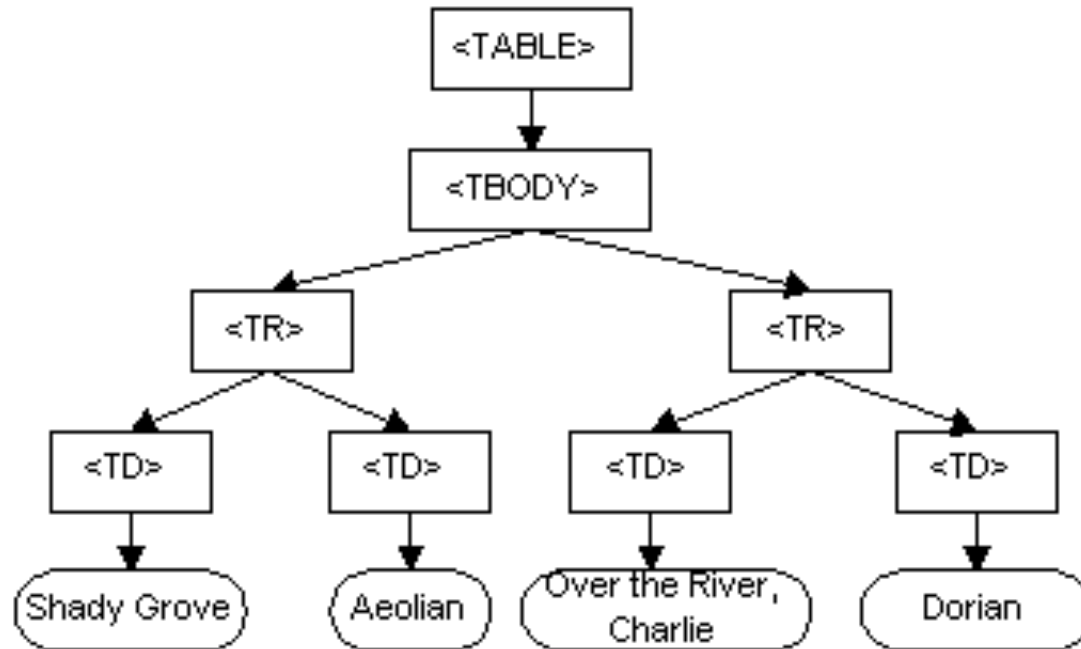
- * évite les erreurs
- * facilite analyse
- * à faire dès l'écriture

Interprétation DOM (2)

- accéder à la deuxième cellule de la deuxième ligne du tableau (Dorian) ?



Interprétation DOM (3)

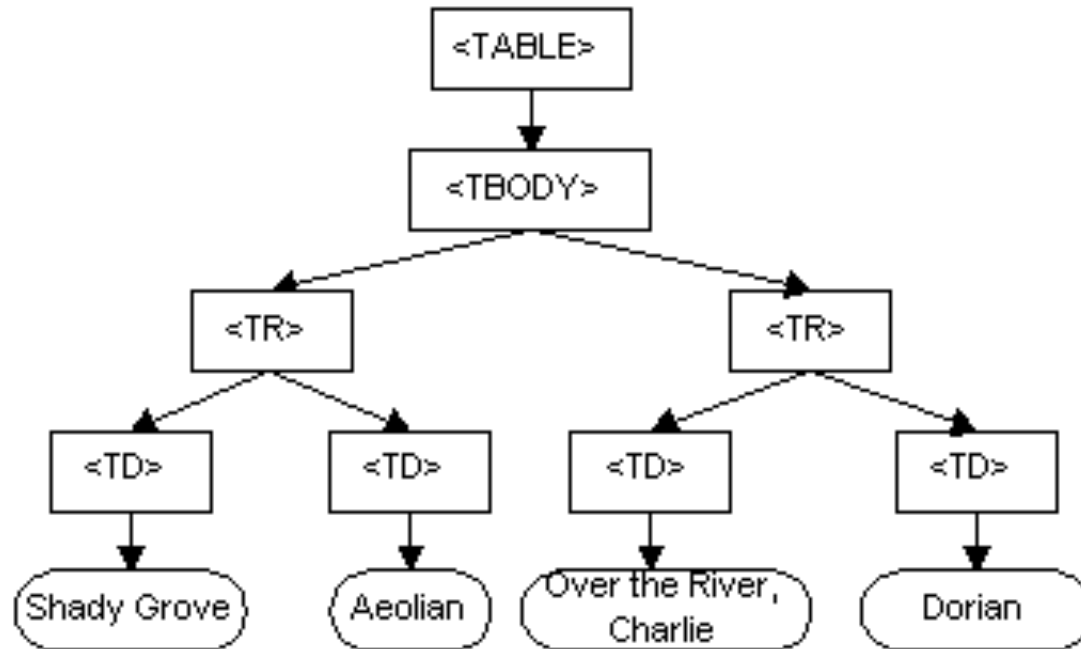


- `document.table[0].tbody[0].tr[1].td[1]`

Interprétation DOM (1bis)

```
<TABLE>
  <TBODY>
    <TR>
      <TD id='cell1' >ShadyGrove</TD>
      <TD id='cell2' >Aeolian</TD>
    </TR>
    <TR>
      <TD id='cell3' >Over the river, Charlie</TD>
      <TD id='cell4' >Dorian</TD>
    </TR>
  </TBODY>
</TABLE>
```

Interprétation DOM (4)



- `document.getElementById("cell4").innerText`

Objets standards du DOM

- DOM → bibliothèques d'objets standards, instanciés automatiquement par le navigateur
- Quelques propriétés standards du DOM : @javascript_innerHTML.htm
 - x.innerHTML ou x.innerHTML – valeur textuelle de l'objet x
 - x.nodeName – nom de l'objet x
 - x.nodeValue – valeur de l'objet x
 - x.attributes – les nœuds attributs de x
 - x.parentNode – le nœud parent de x
 - x.childNodes – les enfants de x
- Méthodes standards du DOM :
 - x.getElementsByTagName(name) – accède à tous les éléments de type name
 - x.getElementById(id) – accède à l'élément id
 - x.removeChild(node) – enlève un fils à x
 - x.appendChild(node) – insert un fils à x

Classes prédéfinies JS

- **Date :**
 - `var nom_objet = new Date()`
 - `nom_objet.getDate()`, `nom_objet.getDay()`...
- **Image :**
 - chargée mais pas affichée, doit venir remplacer le contenu d'un `<IMG>`
 - `src`, `height`, `width`, `complete`
- **Option :**
 - pour les `<SELECT>` des formulaires
 - crée des options supplémentaires qui peuvent ensuite être associées à des `select`
- **Math :**
 - 1 seule instance par page
 - accès aux fonctions : `abs()`, `ceil()`, `random()`, `sqrt()`...
- **String et Array :**
 - `new` optionnel
 - accès aux fonctions : `replace()`, `indexOf()`, `search()`... et `concat()`, `sort()`, `join()`...

JavaScript et CSS

- Possibilité de redéfinir des élément de style CSS directement par le code JavaScript
- Principe : utilisation de la propriété « style » des objets associés aux balises HTML :

```
var e = document.getElementById("divMenu");  
e.style.visibility="hidden";
```

- **Précaution : usage des "-" interdit dans les scripts JavaScript**
 - certaines propriétés de style doivent être transformées :
text-align → textAlign
 - **règle** : le "-" est omis et la lettre suivante est mise en majuscule (CamelCase)

Conclusion

- Une dimension supplémentaire au HTML
- La majorité des objets HTML et des styles CSS modifiables en direct par l'utilisateur :
 - !!! DHTML !!!
- Attention à l'ergonomie, à la sécurité et à la charge CPU du client
- Standardisation du DOM simplifie beaucoup de choses...
- Et au cas où ... `<NOSCRIPT>...</NOSCRIPT>`

CM 9 suite

JavaScript évènements

Gestion des événements (1)

- Objet – événement → action JavaScript
- Sous forme d'une série d'instructions ou d'une fonction
- Chaque objet possède une propriété par événement possible qu'on peut faire pointer sur une séquence d'instructions JavaScript
- Manière de faire :
 - En modifiant la propriété associée à l'évènement de l'objet :
 - `objet.onEvent=fonction_js;`
 - ex : `window.document.links[0].onClick=alert("Un lien cliqué");`

Gestion des événements (5)

```
<HTML>
<HEAD>
</HEAD>
<BODY>
...
  <INPUT type="button" id="myButton" />
...
  <SCRIPT>
    function afficheDial(message){
      alert(message);
    }
    document.getElementById("myButton").onclick=function(){ afficheDial("exemple 1") };
  </SCRIPT>
...
</BODY>
</HTML>
```

Utilisation du DOM

Gestion des événements (2)

- **Attributs de gestion des boutons de la souris :**
 - `onClick` et `onDbClick` permettent la prise en compte d'un clic simple ou double,
 - `onMouseDown` et `onMouseUp` détectent si un bouton de la souris a été enfoncé ou relâché.
- **Attributs de gestion des déplacements de la souris :**
 - `onMouseover` et `onMouseout` détectent si la souris passe sur l'élément courant, ou bien le quitte.
- **Attributs de gestion du clavier :**
 - `onKeyPress`, `onKeyDown` et `onKeyUp` détectent si l'utilisateur a appuyé puis relâché, simplement appuyé, ou relâché une touche du clavier.

Gestion des événements (4)

onFocus/ Blur	Récupération ou perte de focus (=passage de la souris)	onLoad/ Abort	Chargement d'une image ou son interruption
on[Db]Click	Clic de la souris	onUnload	Déchargement d'un document
onMouseDown/ Move/ Out/ Up	Bouton souris enfoncé / mouvement de la souris / retrait de la souris d'un objet / souris sur un objet / bouton relâché	onChange/ Reset/ Select	Changement/annulation de valeurs saisies/Sélection de texte dans un champ de saisie
OnDragDrop	Drag & Drop d'un objet	onSubmit	Soumission d'un formulaire
onMove/ Resize	Déplacement/redimensionnemen t d'une fenêtre	onError	Erreur chargement image ou exécution JS
onKeyDown/ Press/ Up/	Touche enfoncée/pressée/relâchée		

Gestion des événements (1bis)

- Objet – événement → action JavaScript
- Sous forme d'une série d'instructions ou d'une fonction
- Chaque objet possède une propriété par événement possible qu'on peut pointer sur une séquence d'instructions JavaScript
- Deux manières de faire :
 - en modifiant directement les propriétés de l'objet :
 - `objet.onEvent=fonction_js;`
 - ex : `window.document.links[0].onClick=alert("Un lien cliqué");`
 - directement dans la balise :
 - `<balise onEvent="code_js">`
 - ex : `<a href="mapage.htm" onMouseOver="alert("Hello")"> Click ici</a>`

Gestion des événements (5)

```
<HTML>
<HEAD>
  <SCRIPT>
    function afficheDial(message){
      alert(message);
    }
  </SCRIPT>
</HEAD>
<BODY>
...
  <INPUT type="button" onClick="afficheDial('voici un exemple d\'appel de fonctions definie
dans le header!')" />
...
</BODY>
</HTML>
```

Insertion dans les balises HTML

Insertion dans les balises HTML (1)

- Principe : ajout d'attributs associés à du code JavaScript directement dans les balises HTML
 - `<balise onÉvénement="code JavaScript">... </balise>`
 - Gestion d'événements liés aux actions de l'utilisateur:
 - clic de souris (ex "onClick")
 - modification de valeurs saisies par l'utilisateur dans un champ
 - soumission d'un formulaire
 - ...
- Conseillé pour l'appel de fonctions uniquement !
- Astuce : dans ce cas l'objet "balise" sur laquelle on se trouve est accessible dans le code JavaScript par la variable "this"

Gestion des événements (6)

```
<HTML>
<HEAD></HEAD>
<BODY>
  <p onmouseover="this.style.color='blue'" onmouseout="this.style.color='black'">Ceci est un
  texte dont la couleur va changer au passage de la souris.</p>

  <p style="position:absolute; left:10px; top:230px; width:400px"
  onmouseover="this.style.left='100px';this.style.top='300px';this.style.width='600px';this.st
  yle.textAlign='center'"
  onmouseout="this.style.left='10px';this.style.top='230px';this.style.width='400px';this.styl
  e.textAlign='left'">Exemple de texte à géométrie variable.</p>
</BODY>
</HTML>
```

@javascript_events.htm

onmouseover peut être
remplacé par la pseudo-
classe CSS hover

Insertion dans les balises HTML (2)

```
<HTML>
<HEAD>
  <SCRIPT>
    function afficheDial(message){
      alert(message);
    }
  </SCRIPT>
</HEAD>
<BODY>
...
  <INPUT type="button" value="avant" onClick="this.value='apres'; afficheDial('voici un
exemple d'appel de fonctions definie dans le header!')" />
...
</BODY>
</HTML>
```

Insertion dans les balises HTML
Utilisation de this

CM 9 suite

JavaScript exemples En pratique

Quelques exemples

- Quelques exemples illustrant les capacités de Javascript :
 - Changement de propriétés @javascript_journuit.html
 - Menu avec ancrs @javascript_ancres.html
 - Technique du rollover @javascript_roll-over.html
@javascript_clignotant.html
 - Combinaison formulaire HTML @js_radiocheckbox.html
- Attention : comportements peut être encore différents selon les réglages des navigateurs (eg pour ouverture de fenêtres)

Conseils méthodologiques

- Pour cette année préférer partir d'exemples tout fait plutôt que d'écrire complètement une solution
 - beaucoup de détails du langage sont encore complexes à ce niveau (malgré apparente simplicité)
- Recourir aux outils de "debugging" disponibles sur les navigateurs
 - Par exemple Developer Tools avec Chrome, ou Web Developer avec Safari...
 - Penser à écrire sur la console pour vérifier les étapes de vos scripts `console.log('La valeur de i :', i)`
 - Pas simples mais permettent de débloquer quelques situations